identiQ

An project report Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

Artificial Intelligence & Data Science

by

V.Basaveswara Chowdary

2200080133

Under the esteemed guidance of

Mr. Sharath babu
HR
Marketing, Gaint clout technologies pvt ltd

DEPARTMENT OF Artificial Intelligence & Data Science

K L UNIVERSITY

Green Fields, Vaddeswaram, Tadepalli,
Guntur - 522 502, Andhra Pradesh.

2025-26

Gaint clout technologies pvt ltd

Orbit Knowledge City, Raidurg

Hyderabad, Telangana 500019

Bonafide Certificate

This is to certify that this project report entitled **"IdentiQ"** submitted to **Department of Artificial Intelligence & Data Science, KL Deeemed to be University, Guntur,** in connection with the University internship program is a bonafide record of work done by "**V.Basaveswara Chowdary"** under my supervision at the **"**Gaint clout technologies pvt ltd" from 14-07-2025 to 08-10-2025.

Mr. Sharath Babu

HR

K L Deemed to be UNIVERSITY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Certificate

This is Certified that the project entitled **IdentiQ** which is a experimental &/ theoretical &/ Simulation&/ hardware work carried out by VEJANDLA BASAVESWARA CHOWDARY 2200080133, in partial fulfillment for the award of the degree of **Bachelor of Technology** in Department **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**, during the year **2025-26**. The project has been approved as it satisfies the academic requirements.

Department coordinator **Head of the department**

University guide **External**

ACKNOWLEDGEMENT

It is with great pleasure that I express my profound gratitude to our Honorable President, **Sri Koneru Satyanarayana**, for providing the opportunity and necessary facilities that enabled me to successfully complete this internship and project report.

I convey my sincere thanks to **Dr. V. S. V. Prabhakar**, Head of the Department of Artificial Intelligence and Data Science, for his continuous support, motivation, and valuable guidance throughout the course of my academic journey. His leadership has been a constant source of inspiration.

I extend my heartfelt appreciation to my **University Guide, Dr. Venkateswar Rao**, for his insightful suggestions, encouragement, and supervision, which greatly contributed to the successful completion of my project. His guidance helped me bridge the gap between theoretical learning and practical application.

I would also like to express my deep sense of gratitude to my **Company Guide, Mr.Sharath babu**, for his valuable mentorship, expert advice, and constant support during the internship period. His professional experience and guidance provided me with meaningful exposure to industry practices and enriched my learning experience.

I am thankful to the **Regional Manager**, the **Director Of Practice School**, the **Practice School In-Charge**, and all **Department Coordinators** for their assistance and coordination in facilitating the smooth execution of this internship program.

Finally, I wish to acknowledge my gratitude to all faculty members, colleagues, and well-wishers who directly or indirectly contributed to the successful completion of this project report.

IdentiQ

Vejandla Basaveswara Chowdary-2200080133

Kola Sri Varsha-2200080198

Munnangi Janvika-2200031805

ABSTRACT

In the contemporary digital landscape, secure and efficient identification of individuals is crucial across sectors such as security, access control, attendance management, and personalized services. Traditional identification methods, including manual checks and physical tokens, are often prone to errors, time-consuming, and susceptible to misuse. This project introduces a Face Recognition Application—an AI-powered web and mobile solution designed to detect, recognize, and verify individuals using facial biometrics in real time.

The system leverages state-of-the-art computer vision and deep learning techniques to process multimodal inputs, primarily focusing on facial image analysis. Core functionalities include face detection, feature extraction, and matching against a secure database to authenticate or identify users. The application provides a user-friendly interface for face registration, database management, and live recognition, ensuring seamless integration into various workflows.

The project's scope encompasses use cases in enterprise security, educational attendance, smart access systems, and personalized user experiences. Emphasizing privacy, data security, and consent-driven usage, the application is designed to be a responsible and scalable alternative to traditional identification systems.

This report details the problem context, relevant literature, proposed technical methodology, system architecture, key modules, and initial results. The development plan is structured in phases: starting with a minimum viable product (MVP), advancing through enhanced AI integration, and culminating in a robust, full-featured solution. The anticipated outcomes aim to deliver a practical, user-centric, and ethically sound face recognition tool adaptable to diverse industry needs.

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO
1	Introduction	**9 - 12**
2	Literature Review	**13 - 15**
3	Modules And Feature	**16 - 17**
4	Implementation and file structure	**18 - 46**
5	discussion	**47**
6	Conclusion & Future Work	48
7	References	**49**

LIST OF TABLES

3.1 Input Modules

3.2: Feature Categories in Each Module

LIST OF FIGURES

4.4 output of home page

4.5 output of register page

4.6 output of admin-login page

4.7 output of admin dashboard

4.8 output of login-history page

4.9 output of user-management module

List of Symbols, Abbreviations, and Nomenclature

AI - Artificial Intelligence

ML - Machine Learning

CV - Computer Vision

UI - User Interface

API - Application Programming Interface

DB - Database

MVP - Minimum Viable Product

FR - Face Recognition

CNN - Convolutional Neural Network

DBMS - Database Management System

JWT - JSON Web Token

AES-256 - Advanced Encryption Standard (256-bit)

CHAPTER-1
INTRODUCTION

1.1 Background and Motivation

Personal identification and authentication have always been fundamental to human society, influencing areas such as security, access control, and organizational management. In professional and public domains—ranging from workplace attendance and secure facility entry to classroom management and personalized services—the ability to accurately and efficiently verify an individual's identity is critical. Traditionally, identification has relied on physical documents, manual checks, or subjective human recognition, all of which are susceptible to errors, fraud, and inefficiency.

With the rapid advancement of Artificial Intelligence (AI) and Computer Vision (CV), it is now possible to develop systems that can analyze facial features and recognize individuals with high accuracy using computational models. AI-powered face recognition solutions address the limitations of conventional methods by providing contactless, scalable, and automated identification. This is the core motivation behind the Face Recognition App, an intelligent platform designed to detect, recognize, and verify users through facial biometrics in real time.

The motivation for this project also arises from the increasing shift towards digital and automated interactions. As organizations adopt smart attendance systems, secure access solutions, and personalized digital experiences, the demand for reliable and user-friendly identification tools continues to grow. Unlike traditional methods, the Face Recognition App offers a seamless, privacy-conscious, and user-centric approach, focusing on enhancing security, convenience, and operational efficiency. Its purpose is to streamline identification processes, reduce administrative overhead, and foster trust in digital and physical environments.

1.2 Problem Statement

Despite significant advancements in technology, there remains a substantial gap in the availability of reliable, user-friendly, and secure identification tools for everyday use. Current personal identification and authentication methods face the following issues:

1. Limited Accessibility: Traditional identification methods, such as manual ID checks or biometric systems requiring expensive hardware, are often inaccessible to small organizations and individuals.
2. Invasiveness: Many biometric systems require physical contact (e.g., fingerprint scanners), which can cause discomfort and raise hygiene concerns, especially in public or high-traffic environments.
3. Susceptibility to Error and Fraud: Manual verification and physical ID cards are prone to human error, forgery, and misuse, compromising security and efficiency.
4. Lack of Integration: Existing solutions often focus on a single use case (e.g., attendance tracking or access control) and lack seamless integration with other organizational systems.
5. Privacy and Security Concerns: Improper handling of sensitive biometric data can lead to privacy violations and data breaches, raising ethical and legal issues.

Therefore, there is a clear need for a contactless, AI-driven, privacy-conscious face recognition system that can accurately identify and verify individuals in real time. Such a system should be accessible, scalable, and easily integrated into various applications, providing a secure and user-friendly solution for modern identification challenges.

1.3 Objectives

The primary objective of the Face Recognition App project is to design and implement an AI-powered facial identification system that is secure, contactless, and adaptable to various real-world scenarios.

The specific objectives are as follows:

- To develop robust modules for face detection, feature extraction, and face recognition using advanced computer vision techniques.
- To design a secure and scalable database for storing and managing facial data and user profiles.
- To provide real-time identification and verification with clear visual feedback and confidence scores to end users.
- To ensure privacy, data security, and user consent as core design principles throughout the application.
- To create an intuitive admin panel for managing users, monitoring system activity, and configuring access controls.
- To explore practical applications in security, attendance management, access control, and personalized digital experiences.
- To follow a phased development roadmap, beginning with a Minimum Viable Product (MVP) and expanding to a comprehensive web and mobile solution.

1.4 Scope and Limitations

Scope:

- The Face Recognition App will be available as both a web application (for organizations, schools, and businesses) and a mobile app (for on-the-go identification and attendance).
- The system will process facial images and live video streams for real-time face detection and recognition.
- The application will provide identification, verification, attendance tracking, and access control features, with results that can be saved and exported.
- Target users include security personnel, HR managers, school administrators, event organizers, and individuals seeking secure access solutions.
- The app will support user registration, database management, and role-based access controls through an admin panel.
- The system will leverage cloud infrastructure for scalability and offer API endpoints for integration with third-party platforms.

Limitations:

- The system is not intended for use in legal or forensic investigations and should not be relied upon for law enforcement purposes.
- Recognition accuracy may be affected by poor lighting, low camera quality, occlusions (e.g., masks, hats), and changes in appearance.
- The system provides probabilistic identification results, not absolute guarantees; false positives and negatives are possible.
- Ethical usage is limited to consent-based, privacy-respecting scenarios; misuse for surveillance or without user consent is not permitted.
- Variations in demographic representation may impact recognition performance, requiring ongoing dataset updates and bias mitigation.

CHAPTER-2
LITERATURE REVIEW

2.1 Existing Tools and Methods

2.1.1 Manual Identification and ID Cards

Traditional identification methods rely on physical ID cards, badges, or manual verification by security personnel. While widely used, these methods are prone to human error, forgery, and inefficiency, especially in large organizations or public spaces. Lost or stolen IDs can lead to unauthorized access, and manual checks are time-consuming and not scalable.

2.1.2 Fingerprint and Biometric Scanners

Fingerprint and other biometric systems (such as iris or palm scanners) have been adopted for higher security. These systems require physical contact and specialized hardware, which can be costly and raise hygiene concerns. Additionally, biometric spoofing and sensor malfunctions can compromise reliability.

2.1.3 Face Detection and Recognition Algorithms

Recent advances in computer vision and deep learning have enabled the development of robust face detection and recognition algorithms. Libraries such as OpenCV, Dlib, and FaceNet allow for real-time face detection, feature extraction, and matching. These systems are increasingly used in security, attendance management, and personalized user experiences. However, challenges remain in handling variations in lighting, pose, occlusions, and demographic diversity.

2.1.4 Cloud-Based and Mobile Face Recognition Solutions

Cloud-based APIs (e.g., Microsoft Azure Face API, AWS Rekognition, Google Vision) and mobile SDKs have made face recognition more accessible. These solutions offer scalability and integration with existing platforms but raise concerns about data privacy, latency, and dependence on third-party services.

2.2 Gaps in Current Systems

Despite technological progress, existing face recognition systems face several critical shortcomings:

1. Limited Accessibility and Cost: Many advanced systems require expensive hardware or cloud subscriptions, making them inaccessible to small organizations or individuals.
2. Contact-Based and Invasive Methods: Traditional biometrics often require physical contact, which is undesirable in many scenarios.
3. Accuracy and Generalization Issues: Recognition accuracy can degrade due to poor lighting, low camera quality, occlusions (e.g., masks, hats), and changes in appearance.
4. Privacy and Security Concerns: Improper handling of facial data can lead to privacy violations and data breaches. Many systems lack robust consent mechanisms and transparent data usage policies.
5. Lack of Integration and Flexibility: Many solutions are designed for a single use case and do not offer easy integration with other organizational systems or workflows.
6. Interpretability: Most face recognition systems act as “black boxes,” providing a match score without explaining the basis for the decision.

2.3 Research Relevance to the Face Recognition App

The Face Recognition App is designed to address these limitations by providing an accessible, ethical, and user-centric solution. Its contributions over existing systems can be summarized as:

- Contactless and Real-Time Identification: The app leverages advanced computer vision to enable secure, contactless identification and verification.
- User-Centric and Privacy-First Design: The platform emphasizes user consent, data security, and transparent data handling.
- Accessibility: As a web and mobile application, the system is practical for organizations, schools, and individuals without the need for specialized hardware.
- Explainability: The app provides confidence scores and visual feedback, helping users understand the basis for identification results.
- Scalability and Integration: Built with cloud infrastructure and APIs, the system can be integrated into existing workflows such as attendance tracking, access control, and event management.

By addressing these research gaps, the Face Recognition App stands at the intersection of AI research and practical application, contributing to the advancement of secure, ethical, and user-friendly identification technologies.

Chapter 3

MODULES AND FEATURES

The system is divided into three major categories of modules: Input Modules, Processing Modules, and Output Modules. Each module is designed to operate independently but integrates seamlessly through the Face Recognition Engine.

3.1 Input Modules

Module Type	Module Name	Description	Output Example
Input	Image Input	Captures static images for face analysis	Image file (JPEG/PNG)
	Video Input	Records live video streams for real-time analysis	Video frames
	User Data Input	Accepts user details for registration	Name, ID, metadata

Table 3.1: Module Overview

3.2 Processing Modules

- Preprocessing Unit: Cleans and normalizes image/video data, adjusts lighting, and prepares for feature extraction.
- Feature Extraction Unit: Extracts facial landmarks, embeddings, and other biometric features.
- Face Recognition Engine: Matches extracted features against the database to identify or verify the user.

Module	Features Extracted	Tools/Techniques
Face Detection	Face location, bounding boxes	OpenCV, Dlib
Feature Extraction	Facial landmarks, embeddings	FaceNet, DeepFace
Recognition	Match score, user identity	Cosine similarity, KNN, SVM

Table 3.2: Feature Categories in Each Module

3.3 Output Modules

- Recognition Result Display: Shows identification/verification result with user info and match score.
- Confidence Level: Provides model confidence to inform users about reliability.
- Detailed Report: Includes logs of recognition attempts, timestamps, and user details.
- Export Functionality: Generates PDF or CSV reports for attendance, access logs, or audit purposes.

Chapter 4

IMPLEMENTATION AND FILE STRUCTURE

4.1 Implementation Overview

The frontend of the Face Recognition App is developed using React.js and styled with Tailwind CSS. It provides a responsive and user-friendly interface for capturing images or video, registering users, and displaying recognition results. The frontend communicates with the backend API for processing and data retrieval.

Key features implemented in the frontend include:

- User registration and login
- Image/video capture for face recognition
- Real-time display of recognition results and confidence scores
- User and admin dashboards for managing profiles and viewing logs
- Export and download of attendance or access reports

4.2 File Structure

Below is a typical file structure for the frontend React application:

```
face-recognition-app/
    ├── src/
    │   ├── components/
    │   │   ├── WebcamCapture.jsx     # Webcam access and photo
capture
    │   │   ├── FaceRecognition.jsx   # Face detection display
    │   │   ├── Navigation.jsx        # Navigation bar
    │   │   └── AuthForm.jsx          # Reusable login/register form
    │   │
    │   ├── pages/
    │   │   ├── Home.jsx              # Landing page
```

```
│   │   ├── Login.jsx          # User login
│   │   ├── Register.jsx       # User registration
│   │   ├── Dashboard.jsx      # Main user dashboard
│   │   └── LoginHistory.jsx   # Login attempts history
│   │
│   ├── routes/
│   │   └── router.jsx         # Route configurations
│   │
│   ├── utils/
│   │   └── api.js             # API client functions
│   │
│   ├── App.jsx                # Root component
│   └── main.jsx               # Entry point
│
├── public/
│   └── index.html             # HTML template
│
├── server/
│   ├── controllers/
│   │   ├── authController.js  # Authentication logic
│   │   └── faceController.js  # Face recognition logic
│   │
│   ├── models/
│   │   ├── User.js            # User data schema
│   │   └── Log.js             # Login logs schema
│   │
│   ├── routes/
│   │   ├── auth.js            # Auth endpoints
│   │   └── face.js            # Face recognition endpoints
```

```
│   │
│   └── server.js            # Express server setup
│
├── .env                     # Environment variables
├── package.json             # Dependencies
└── vite.config.js           # Vite configuration
```

4.3 Key Files and Directories

- components/: Contains reusable UI elements such as Camera, FaceRecognitionResult, Navbar, and Modal components.
- pages/: Includes main views like Home, Register, Login, Dashboard, and Reports.
- services/: Handles API requests to the backend for user management and face recognition.
- hooks/: Custom hooks for camera access, authentication, and state management.
- styles/: Tailwind CSS and any additional custom styles.
- App.js: Sets up routing and global layout.
- index.js: Renders the React app to the DOM.

Main.jsx code:

```
import React from "react";
import ReactDOM from "react-dom/client";
import { RouterProvider } from "react-router-dom";
import router from "./routes/router";
import "../styles/index.css"; // optional styling

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
);
```

App.jsx code:

```
import { Outlet, Link } from "react-router-dom";

export default function App() {
  return (
    <div className="p-4">
      <nav className="navbar">
        <div className="navbar-brand">FaceRecognition</div>
        <div className="navbar-links">
          <Link to="/" className="nav-link">Home</Link>
          <Link to="/login" className="nav-link">Login</Link>
          <Link to="/register" className="nav-link">Register</Link>
          <Link to="/dashboard" className="nav-link">Dashboard</Link>
        </div>
```

```
      </nav>
      <Outlet />
    </div>
  );
}
```

Router.jsx code:

```
import { createBrowserRouter } from "react-router-dom";
import App from "../App";
import Home from "../pages/home";
import Login from "../pages/login";
import Dashboard from "../pages/dashboard";
import UserManagement from "../pages/user-management";
import Register from "../pages/register";
import LoginHistory from "../pages/LoginHistory";
import WelcomePage  from "../pages/welcomepage";
import AdminLogin from "../pages/admin-login";

import BackendLogs from "../pages/backend-logs";
import LiveFaceRecognition from "../../src/pages/LiveFaceRecognition";



const router = createBrowserRouter([
  {
    path: "/",
    element: <App />,
    children: [
      {
        index: true,
```

```
  element: <Home />,
},
{
  path: "login",
  element: <Login />,
},
{
  path: "register",
  element: <Register />,
},
{
  path: "dashboard",
  element: <Dashboard />,
},
{
  path: "login-history",
  element: <LoginHistory />,
},
{
  path: "admin-login",
  element: <AdminLogin />,
},
{
  path: "backend-logs",
  element: <BackendLogs />,
},
{
  path: "welcomepage",
  element: <WelcomePage />,
```

```
      },
      {
        path: "live-face-recognition",
        element: <LiveFaceRecognition />,
      },
      {
        path: "user-management",
        element: <UserManagement />,
      },




    ]
  }
]);

export default router;
```

home.jsx code:

```
import React from "react";
import { Link } from "react-router-dom";
import "./home.css";

function Home() {
  return (
    <div className="container">
      <h1>IdentiQ</h1>
```

```
      <p>Welcome to the Face Recognition App. This application allows you
to scan and recognize faces.</p>
      <div className="button-container">
        <Link to="/login" className="button">
          Login
        </Link>
        <Link to="/register" className="button">
          Register a new user
        </Link>
        <Link to="/admin-login" className="button">
          Admin Login
        </Link>
      </div>
    </div>

  );
}


export default Home;
```

WELCOME-PAGE.JSX CODE:

```
import React, { useEffect, useState } from "react";
import { useLocation, Link } from "react-router-dom";

function WelcomePage() {
  const location = useLocation();
  const userName = location.state?.userName || "User";
  const [seconds, setSeconds] = useState(5);

  useEffect(() => {
    if (seconds <= 0) {
      window.location.href = "https://mail.google.com";
      return;
    }
    const id = setInterval(() => setSeconds((s) => s - 1), 1000);
    return () => clearInterval(id);
  }, [seconds]);

  return (
    <div className="container">
      <h2 style={{ color: "#a78bfa", fontWeight: 900, marginBottom: "2rem" }}>
        Welcome, {userName}!
      </h2>
      <p>
        Redirecting to Gmail in <strong>{seconds}</strong> second{seconds !== 1 ? "s" :
""}.
      </p>
      <Link to="/" className="button secondary">
        Back to Home
      </Link>
    </div>
  );
}
```

```
export default WelcomePage;
```

USER-MANAGEMENT.JSX CODE:

```
import React, { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import "./user-management.css";

const normalizeUser = (item, key) => ({
  id: item.id || item._id || null,
  name: item.name || item.userName || item.username || item.fullName || "",
  email: item.email || item.mail || "",
  role: item.role || item.userRole || "user",
  loginTime: item.loginTime || item.loginTimestamp || item.lastLogin || null,
  image: item.image || item.avatar || null,
  _rawKey: key || ""
});

const loadAllUsersFromLocalStorage = () => {
  try {
    const result = [];

    // Prefer a single "users" array key
    const stored = localStorage.getItem("users");
    if (stored) {
      try {
        const parsed = JSON.parse(stored);
        if (Array.isArray(parsed)) return parsed.map((p) => normalizeUser(p, "users"));
        if (parsed && typeof parsed === "object") return Object.values(parsed).map((p) =>
normalizeUser(p, "users"));
      } catch (e) {
```

```
      // ignore
    }
  }

  // Otherwise scan keys
  for (let i = 0; i < localStorage.length; i++) {
    const key = localStorage.key(i);
    if (!key) continue;
    const lower = key.toLowerCase();
    if (!(lower === "user" || lower.startsWith("user"))) continue;
    const raw = localStorage.getItem(key);
    if (!raw) continue;
    try {
      const parsed = JSON.parse(raw);
      if (parsed && typeof parsed === "object") result.push(normalizeUser(parsed, key));
    } catch (e) {
      // ignore non-json
    }
  }

  // dedupe
  const seen = new Set();
  const dedup = [];
  for (const u of result) {
    const id = `${u.name}|${u.email}|${u.loginTime}`;
    if (!seen.has(id)) {
      seen.add(id);
      dedup.push(u);
    }
  }

  return dedup;
} catch (err) {
  console.error(err);
```

```
    return [];
  }
};

export default function UserManagement() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  // Vite exposes env variables via import.meta.env and requires VITE_ prefix for client
use
  const API_USERS = import.meta.env.VITE_API_USERS || "/api/users";

  const fetchUsersFromServer = async () => {
    setLoading(true);
    setError(null);
    try {
      // Configurable via Vite envs
      const AUTH_HEADER = import.meta.env.VITE_API_AUTH_HEADER ||
"Authorization";
      const AUTH_KEY = import.meta.env.VITE_API_AUTH_KEY || null;
      const USE_CREDENTIALS = (import.meta.env.VITE_API_USE_CREDENTIALS ||
"false") === "true";

      // Build headers (token from localStorage or static key)
      const token = localStorage.getItem("token") || localStorage.getItem("authToken") ||
localStorage.getItem("accessToken");
      const headers = { "Content-Type": "application/json" };
      if (AUTH_KEY) {
        headers[AUTH_HEADER] = AUTH_KEY;
      } else if (token) {
        headers[AUTH_HEADER] = AUTH_HEADER.toLowerCase() === "authorization"
? `Bearer ${token}` : token;
```

```
    }

    const fetchOpts = { headers };
    if (USE_CREDENTIALS) fetchOpts.credentials = "include";

    const res = await fetch(API_USERS, fetchOpts);
    if (res.status === 401 || res.status === 403) {
      // unauthorized - redirect to admin-login
      navigate("/admin-login");
      return;
    }
    if (!res.ok) {
      const text = await res.text().catch(() => res.statusText);
      throw new Error(text || `Status ${res.status}`);
    }

    // Ensure JSON
    const contentType = (res.headers.get("content-type") || "").toLowerCase();
    let data;
    if (!contentType.includes("application/json")) {
      const text = await res.text().catch(() => "");
      throw new Error(text ? text : `Expected JSON but got ${contentType || 'unknown'}`);
    } else {
      data = await res.json();
    }

    // Accept multiple shapes: top-level array, or wrapper objects like { users: [...] }, {
data: [...] }
    let usersArray = [];
    if (Array.isArray(data)) usersArray = data;
    else if (data && typeof data === 'object') {
      const candidates = ["users", "data", "results", "rows", "items"];
      for (const c of candidates) {
        if (Array.isArray(data[c])) { usersArray = data[c]; break; }
```

```
      }
      if (!usersArray.length) {
        for (const k of Object.keys(data)) {
          if (Array.isArray(data[k])) { usersArray = data[k]; break; }
        }
      }
    }

    const list = Array.isArray(usersArray) ? usersArray.map(u => ({
      id: u.id || u._id || null,
      name: u.name || u.userName || u.username || u.fullName || "",
      email: u.email || u.mail || u.contact || "",
      role: u.role || u.userRole || (u.isAdmin ? "admin" : "user") || "user",
      loginTime: u.loginTime || u.lastLogin || u.loginTimestamp || u.last_seen || u.lastLogin
|| null,
      image: u.image || u.avatar || u.photo || null,
      _rawKey: u.id || u._id || u.email || "server"
    })) : [];

    setUsers(list);
  } catch (err) {
    console.error("Failed to fetch users from server:", err);
    const message = err && err.message ? err.message : "Failed to load users from server";
    setError(message);

    // fallback to local cache
    try {
      const local = loadAllUsersFromLocalStorage();
      if (Array.isArray(local) && local.length > 0) {
        setUsers(local);
        setError(prev => (prev ? prev + " — loaded local cache (click Refresh to retry
server)" : "Loaded local cache"));
      }
    } catch (e) {
```

```
      // ignore
    }
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  // try server first, then page UI allows fallback
  fetchUsersFromServer();
}, []);

return (
  <div className="user-management">
    <div className="um-header">
      <h2>User Management</h2>
      <div className="um-actions">
        <button onClick={() => fetchUsersFromServer()}>Refresh (server)</button>
        <button onClick={() => setUsers(loadAllUsersFromLocalStorage())}>Load local
cache</button>
        <button onClick={() => navigate(-1)}>Back</button>
      </div>
    </div>

    {loading ? (
      <p className="um-loading">Loading users...</p>
    ) : null}

    {/* Compact notices: don't display raw server HTML or long error text */}
    {error && users.length > 0 && (
      <div className="um-note">
        <em>Showing local cache (server unavailable). Click "Refresh (server)" to
retry.</em>
      </div>
```

```
      )}

      {error && users.length === 0 && (
        <div className="um-error">
          <strong>Unable to load users from server.</strong>
          <div className="um-error-sub">Try clicking "Load local cache" or check your
backend.</div>
        </div>
      )}

      <p className="um-count">Total users: {users.length}</p>

      {users.length === 0 ? (
        <p className="um-empty">No users found.</p>
      ) : (
        <table className="um-table">
          <thead>
            <tr>
              <th>#</th>
              <th>Name</th>
              <th>Email</th>
              <th>Role</th>
              <th>Last Login</th>
              <th>Photo</th>
              <th>Source</th>
              <th></th>
            </tr>
          </thead>
          <tbody>
            {users.map((u, i) => (
              <tr key={i}>
                <td className="um-cell-index">{i+1}</td>
                <td>{u.name || "—"}</td>
                <td>{u.email || "—"}</td>
```

```
            <td>{u.role || "user"}</td>
            <td>{u.loginTime ? new Date(u.loginTime).toLocaleString() : "—"}</td>
            <td>
              {u.image ? (
                <img src={u.image} alt="avatar" className="um-avatar-img" />
              ) : (
                <div className="um-avatar-
placeholder">{(u.name||"?").charAt(0).toUpperCase()}</div>
              )}
            </td>
            <td>{u._rawKey || "server"}</td>
            <td>
              {u.id ? (
                <button
                  className="btn-delete"
                  onClick={async () => {
                    if (!confirm(`Delete user ${u.name || u.email || u.id}? This cannot be
undone.`)) return;
                    try {
                      const token = localStorage.getItem("token") ||
localStorage.getItem("authToken") || localStorage.getItem("accessToken");
                      const headers = { "Content-Type": "application/json" };
                      if (token) headers["Authorization"] = `Bearer ${token}`;
                      const delRes = await fetch(`${API_USERS}/${u.id}`, { method:
'DELETE', headers });
                      if (!delRes.ok) {
                        const txt = await delRes.text().catch(() => delRes.statusText);
                        alert(`Delete failed: ${txt}`);
                        return;
                      }
                      // remove from UI
                      setUsers(prev => prev.filter(x => x.id !== u.id));
                    } catch (e) {
                      console.error(e);
```

```
                    alert('Delete failed');
                  }
                }}
              >
                Delete
              </button>
            ) : null}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
 )}

  </div>
 );
}
```

INDEX.CSS CODE:

```
/* Modern dark gradient background */
body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
  color: #e5e7fa;
  background: radial-gradient(ellipse at 60% 40%, #23264a 60%, #181c2f 100%);
  min-height: 100vh;
}

/* Attractive Navigation Bar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
```

```
  background: rgba(36, 39, 66, 0.95);
  padding: 1rem 2rem;
  border-radius: 12px;
  box-shadow: 0 4px 24px 0 rgba(31, 38, 135, 0.12);
  margin-bottom: 2rem;
}

.navbar-brand {
  font-size: 1.5rem;
  font-weight: bold;
  color: #a78bfa;
  letter-spacing: 1px;
}

.navbar-links {
  display: flex;
  gap: 1.5rem;
}

.nav-link {
  color: #e5e7fa;
  text-decoration: none;
  font-weight: 500;
  font-size: 1.07rem;
  padding: 0.3rem 0.8rem;
  border-radius: 6px;
  transition: background 0.2s, color 0.2s;
}

.nav-link:hover, .nav-link.active {
  background: #7c3aed;
  color: #fff;
}
```

```
/* Card-like container styling */
.container {
  max-width: 420px;
  margin: 48px auto;
  padding: 2.5rem 2rem 2rem 2rem;
  background: rgba(24, 28, 47, 0.98);
  border-radius: 18px;
  box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.25);
  text-align: center;
  position: relative;
  z-index: 1;
}

h1 {
  margin-bottom: 1rem;
  color: #a78bfa;
}

p {
  margin-bottom: 1.5rem;
  color: #b0b3c7;
}

.button-container {
  display: flex;
  justify-content: center;
  gap: 1rem;
  margin-top: 2rem;
}

.button {
  display: inline-block;
  padding: 0.75rem 1.5rem;
  background-color: #7c3aed;
```

```
  color: white;
  text-decoration: none;
  border-radius: 4px;
  font-weight: bold;
  transition: background-color 0.3s;
}

.button:hover {
  background-color: #6366f1;
}

.button.secondary {
  background-color: #95a5a6;
}

.button.secondary:hover {
  background-color: #7f8c8d;
}

.camera-placeholder {
  width: 100%;
  max-width: 350px;
  height: 180px;
  margin: 2rem auto;
  background-color: #23264a;
  color: #b0b3c7;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 8px;
  border: 1.5px dashed #2e335a;
}

.result-container {
```

```
  margin: 2rem 0;
}

.result-item {
  background-color: #23264a;
  padding: 1.5rem;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(31, 38, 135, 0.08);
  margin-bottom: 1rem;
}

.result-item h2 {
  color: #a78bfa;
  margin-bottom: 0.5rem;
}

.result-item p {
  margin-bottom: 0.5rem;
  color: #b0b3c7;
}

/* Login page specific styles */
.status-message {
  margin: 1rem 0;
  padding: 0.75rem;
  background-color: #23264a;
  border-radius: 4px;
  border-left: 4px solid #7c3aed;
  color: #b0b3c7;
  transition: all 0.3s ease;
}

.status-message.success {
  background-color: #23264a;
```

```
  border-left: 4px solid #28a745;
  color: #28e07a;
}

.status-message.error {
  background-color: #23264a;
  border-left: 4px solid #dc3545;
  color: #ff6b81;
}

.status-message.scanning {
  background-color: #23264a;
  border-left: 4px solid #ffc107;
  color: #ffc107;
  animation: pulse 1.5s infinite;
}

@keyframes pulse {
  0% {
    opacity: 0.7;
  }
  50% {
    opacity: 1;
  }
  100% {
    opacity: 0.7;
  }
}

.navigation-options {
  margin-top: 1rem;
  color: #b0b3c7;
}
```

```
.input {
  padding: 10px;
  border-radius: 5px;
  border: 1px solid #2e335a;
  background: #23264a;
  color: #e5e7fa;
  font-size: 1rem;
  outline: none;
  margin-bottom: 0;
  transition: border 0.2s;
}
.input:focus {
  border: 1.5px solid #7c3aed;
}
.photo-flex {
  display: flex;
  flex-direction: column;      /* Stack vertically */
  align-items: center;
  gap: 0.7rem;                 /* Space between photo and button */
  justify-content: center;
  margin-bottom: 1rem;
}

.photo-preview {
  width: 140px;
  height: 110px;
  object-fit: cover;
  border-radius: 8px;
  background: #23264a;
}

.retake-btn {
  background: #7c3aed;
  color: #fff;
```

```
  border: none;
  border-radius: 4px;
  padding: 0.5rem 1rem;
  font-weight: 500;
  cursor: pointer;
  transition: background 0.2s;
  margin-top: 0;
}

.retake-btn:hover {
  background: #6366f1;
}
```

Welcomepage.jsx

```
import React, { useEffect, useState } from "react";
import { useLocation, Link } from "react-router-dom";

function WelcomePage() {
  const location = useLocation();
  const userName = location.state?.userName || "User";
  const [seconds, setSeconds] = useState(5);

  useEffect(() => {
    if (seconds <= 0) {
      window.location.href = "https://mail.google.com";
      return;
    }
    const id = setInterval(() => setSeconds((s) => s - 1), 1000);
    return () => clearInterval(id);
  }, [seconds]);
```

```
  return (
    <div className="container">
      <h2 style={{ color: "#a78bfa", fontWeight: 900, marginBottom: "2rem"
}}>
        Welcome, {userName}!
      </h2>
      <p>
        Redirecting to Gmail in <strong>{seconds}</strong> second{seconds
!== 1 ? "s" : ""}.
      </p>
      <Link to="/" className="button secondary">
        Back to Home
      </Link>
    </div>
  );
}

export default WelcomePage;
```

These are the main codes that are used to implement my project

Outputs are projected below

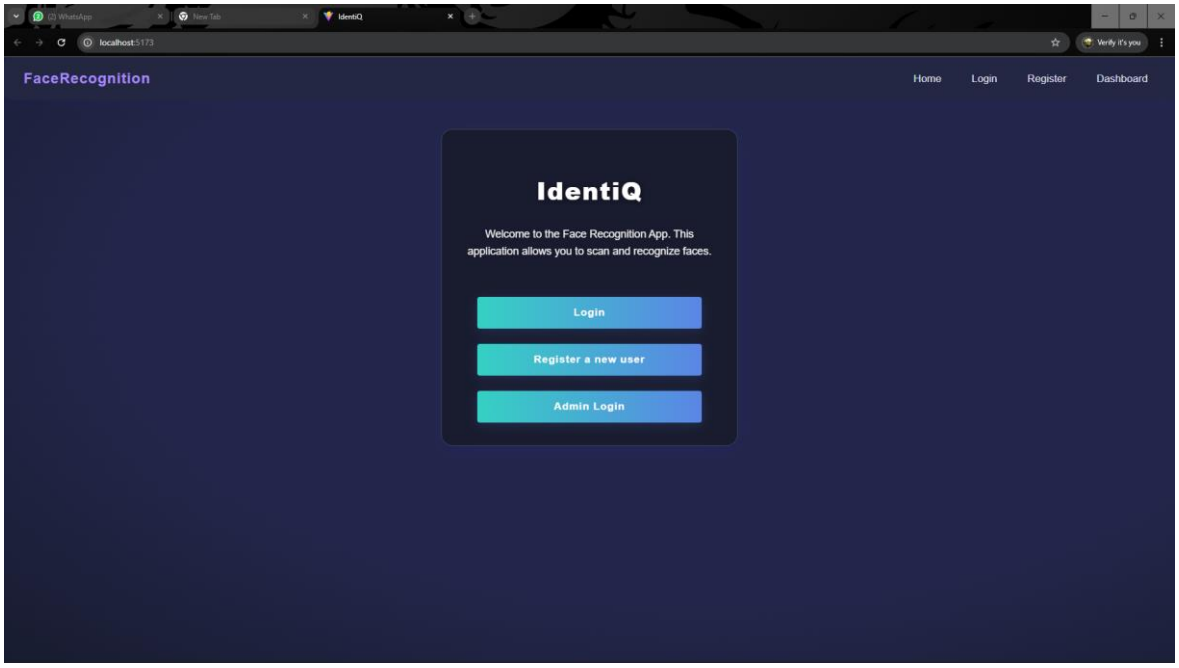

Figure4.4 output of home page

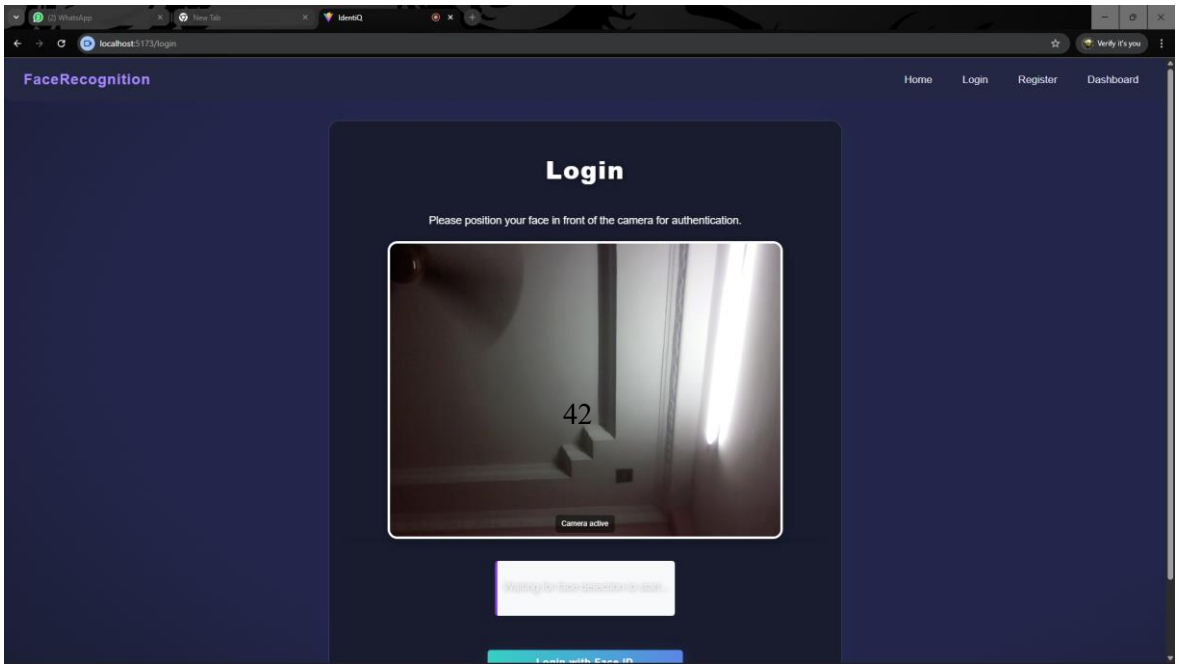

Figure 4.5 output of login page

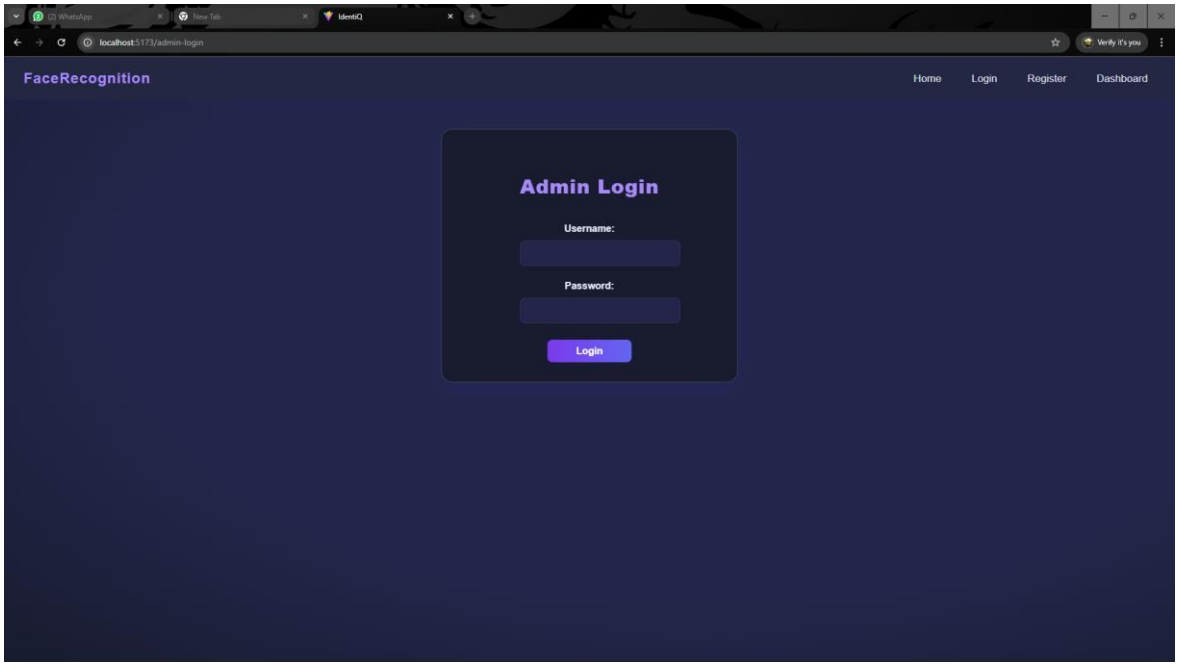

Figure 4.6 output of admin login page

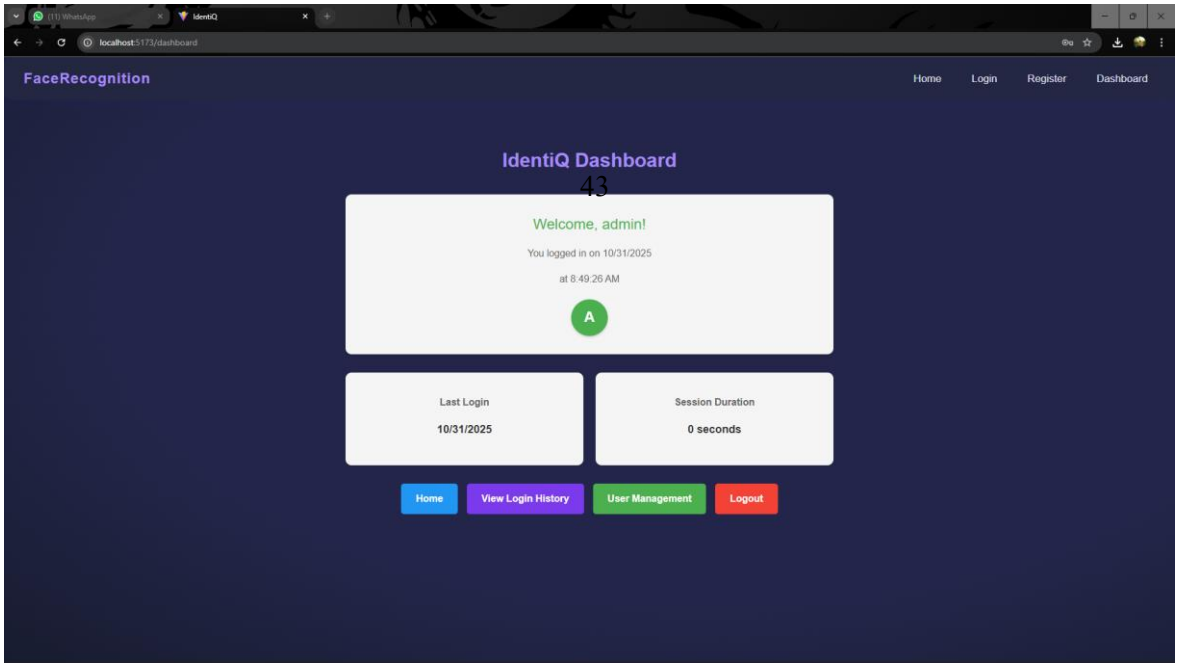

Figure 4.7 it displays the dashboard of admin login page

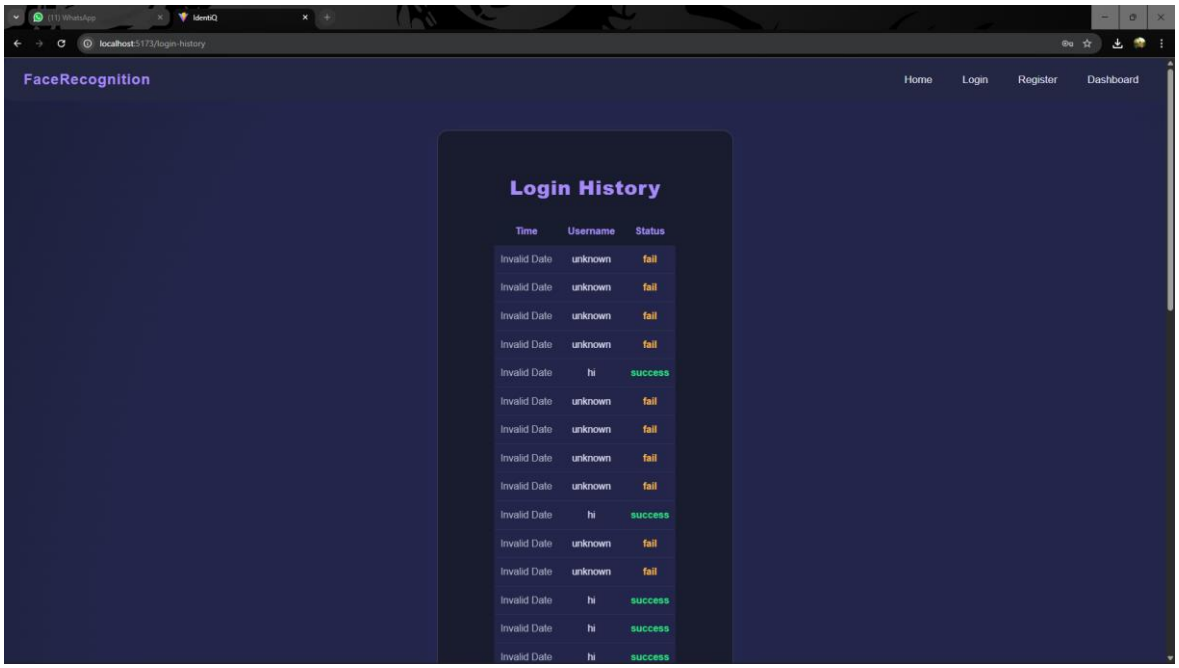

Figure 4.8 login history page that displays the user's login history

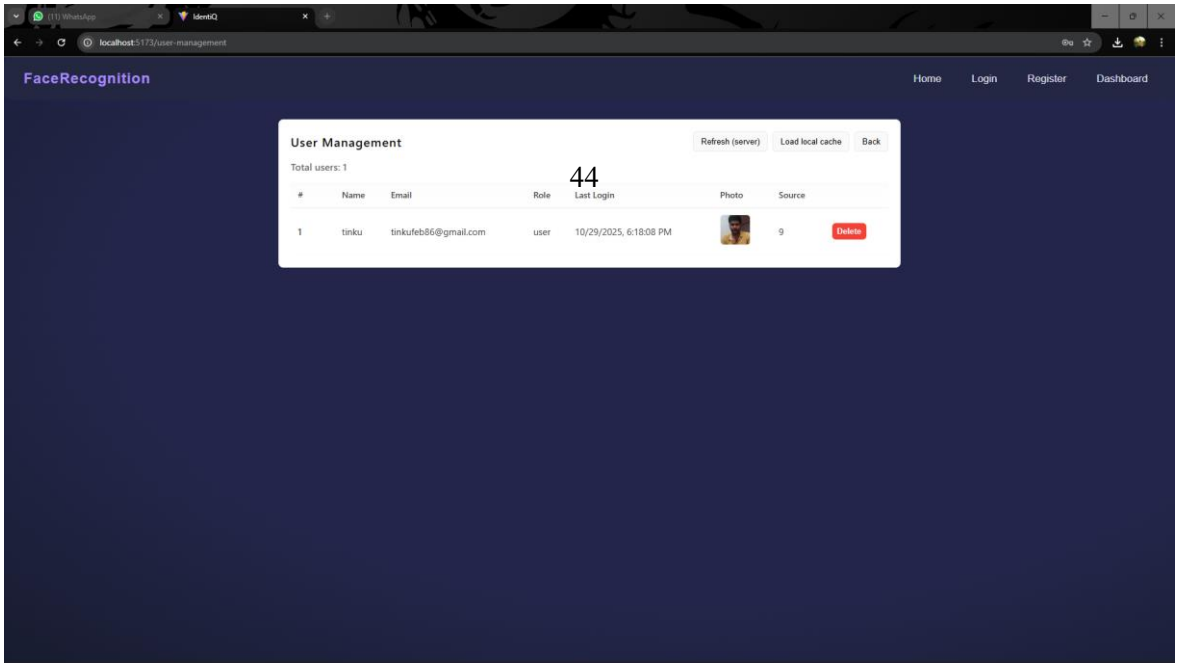

Figure 4.9 it displays the user management module from here we can manage the user data such as past login user photo we also included the CRUD operation

Chapter 5
DISCUSSION

The Face Recognition App processes sensitive biometric data, including facial images and video streams, for identification and verification. While these capabilities offer significant benefits in security and convenience, they also introduce important privacy, ethical, and legal considerations. This chapter outlines how the Face Recognition App ensures responsible development and deployment.

5.1 Privacy Considerations

Consent First: Users must provide explicit consent before any facial data is captured or processed.

Right to Delete: Users have the ability to permanently delete their profiles, facial data, and activity logs from the system.

Data Security: All biometric data is encrypted using AES-256 for storage and protected with TLS 1.3 during transmission.

Minimal Retention: Only essential metadata (e.g., recognition logs, timestamps) is retained; raw images and video can be automatically deleted after processing.

5.2 Ethical Considerations

Not for Surveillance: The system clearly states it is not intended for covert surveillance or law enforcement use.

Bias Mitigation: Training datasets include diverse faces across age, gender, and ethnicity to minimize algorithmic bias.

Transparency: Users are shown match scores and confidence levels to avoid misinterpretation of results as absolute identification.

Responsible Use: The app is recommended for secure access, attendance, and user authentication in organizations, not for criminal investigations or unauthorized monitoring.

Chapter 6
CONCLUSION AND FUTURE WORK Conclusion

Conclusion

Through the development of the Face Recognition App, I had demonstrated how AI-powered facial identification can be transformed into a practical, secure, and user-friendly solution for modern authentication needs. The integration of real-time face detection, feature extraction, and recognition modules provides a seamless experience for users and administrators alike. The project emphasizes privacy, consent, and transparency, ensuring that biometric data is handled responsibly and ethically.

By exploring the system's architecture and early results, you recognize the importance of robust preprocessing, diverse training data, and clear confidence indicators in building user trust. While initial deployments show promising accuracy and usability, further validation and optimization are needed to ensure consistent performance across varied environments and demographics.

This phase establishes a strong foundation for a scalable and adaptable face recognition platform, balancing technological innovation with ethical responsibility. The groundwork laid here prepares the system for future enhancements and broader adoption.

Future Work

Enhance real-time recognition accuracy under diverse lighting, backgrounds, and camera qualities.

Expand dataset diversity to improve performance across different ages, ethnicities, and facial features.

Improve explainability by providing users with more detailed feedback on recognition decisions.

Integrate additional biometric modalities (e.g., voice, gait) for multi-factor authentication.

Develop advanced privacy features such as on-device processing and federated learning.

Chapter 7

REFERENCES

REFERENCES

1. Taigman, Y., Yang, M., Ranzato, M., & Wolf, L. (2014). DeepFace: Closing the Gap to Human-Level Performance in Face Verification. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
2. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
3. King, D. E. (2009). Dlib-ml: A Machine Learning Toolkit. *Journal of Machine Learning Research*, 10, 1755-1758.
4. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
5. Parkhi, O. M., Vedaldi, A., & Zisserman, A. (2015). Deep Face Recognition. *British Machine Vision Conference (BMVC)*.
6. Microsoft Azure. (2023). Face API Documentation. https://learn.microsoft.com/en-us/azure/cognitive-services/face/
7. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
8. European Union. (2016). General Data Protection Regulation (GDPR). https://gdpr.eu/
9. Government of India. (2023). Digital Personal Data Protection Act (DPDP).
10. Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks. *IEEE Signal Processing Letters*, 23(10), 1499-1503.